PROJEKTDOKUMENTATION

Gruber Login

Login-Anwendung mit FastAPI, MySQL und Docker

Stand: 22. Oktober 2026

1. Ziel des Projekts

Eine Webanwendung für Registrierung und Anmeldung, bei der Nutzer selbst wählen, wie stark ihr Login abgesichert ist. Das Projekt zeigt den Umgang mit Passwort-Hashing, Zwei-Faktor-Authentifizierung, Sessions und containerisiertem Betrieb. Der Code bleibt bewusst klein und ist in klar getrennte Module aufgeteilt, damit jeder Ablauf nachvollziehbar ist.

2. Funktionsumfang

Modus	Ablauf
Passwort	E-Mail und Passwort, danach direkt angemeldet.
E-Mail-Code	Passwort und sechsstelliger Code per E-Mail (10 Minuten gültig).
Authenticator (TOTP)	Passwort und Code aus einer Authenticator-App. Einrichtung per QR-Code (RFC 6238); der erste Code bestätigt die Einrichtung und meldet direkt an.

Weitere Funktionen:

- Landingpage mit fünf Ansichten: Anmelden, Konto erstellen, Code bestätigen, Authenticator einrichten, Passwort zurücksetzen.
- Geschützte Willkommensseite (`/welcome`) mit Sitzungsdaten und Abmeldung.
- Passwort-Reset per E-Mail-Code (15 Minuten gültig).
- Schutz vor Durchprobieren: begrenzte Fehlversuche, Rate-Limits pro IP und E-Mail, TOTP-Replay-Schutz.
- Optionale zeitliche Login-Beschränkung, zum Beispiel 6 bis 22 Uhr.
- 30 automatisierte Tests und eine Lint-Prüfung mit ruff.

3. Technischer Aufbau

Komponente	Technik	Aufgabe
Backend	Python, FastAPI, SQLAlchemy (`app/`)	Seiten ausliefern, API, Sessions und Fachlogik
Frontend	HTML, CSS, JavaScript (`app/templates/`, `app/static/`)	Landingpage, Formulare, QR-Code-Anzeige
Datenbank	MySQL 8.4	Tabellen `users`, `sessions`, `email_otps`
Zugang	Cloudflare Tunnel (cloudflared)	HTTPS ohne Portfreigabe
Betrieb	Docker Compose	Start mit `docker compose up -d --build db web`

Das Backend ist ein Python-Paket. Jede Datei hat eine Aufgabe:

```
app/
├── __init__.py        App-Factory, Sicherheits-Header, Fehler-Handler
├── config.py          Grenzwerte, Umgebungsvariablen, SMTP
├── db.py              Engine, Session pro Request, Schema-Upgrade
├── models.py          Tabellen users, sessions, email_otps
├── security.py        Argon2id, HMAC, Verschlüsselung
├── mailer.py          SMTP-Versand oder Dev-Modus
├── ratelimit.py       In-Memory-Limiter (gleitendes Fenster)
├── schemas.py         Request-Bodies (Pydantic)
├── errors.py          AppError / RateLimited
├── validation.py      E-Mail normalisieren und prüfen
├── routers/           HTTP-Schicht: pages, auth, password, system
├── services/          Regeln: accounts, sessions, email_otp, totp, throttling
├── templates/         index.html, welcome.html
└── static/            style.css, app.js, welcome.js
```

Die Aufteilung folgt einer einfachen Regel: **Router** kennen HTTP, **Services** kennen die Regeln, **Basismodule** wie `config.py` oder `security.py` kennen nur ihr Thema. Fehler wirft die Fachlogik als `AppError`; ein zentraler Handler in `app/__init__.py` macht daraus eine JSON-Antwort.

4. Ablauf der Anmeldung

1. Der Browser sendet E-Mail und Passwort als JSON an `POST /api/login`. Die Übertragung erfolgt über HTTPS.
2. Der Server sucht das Konto und prüft das Passwort gegen den gespeicherten Argon2id-Hash. Bei unbekannter E-Mail wird ein Dummy-Hash gerechnet, damit die Antwortzeit nichts verrät.
3. Ist der Modus „Passwort“, wird direkt eine Session angelegt. Sonst antwortet der Server mit `2fa_required`.
4. Der zweite Faktor wird über `POST /api/verify-2fa` geprüft: E-Mail-Code oder TOTP-Code aus der App.
5. Nach erfolgreicher Prüfung entsteht ein zufälliger Session-Token. In der Datenbank liegt nur sein SHA-256-Hash, im Browser ein `HttpOnly`-Cookie mit sieben Tagen Laufzeit.
6. `/welcome` prüft das Cookie bei jedem Aufruf gegen die Datenbank. Ohne gültige Session wird zurück zur Startseite geleitet.

> **Registrierung mit Authenticator-App:** Der Server erzeugt ein TOTP-Secret, verschlüsselt es für die Datenbank und liefert ein SVG mit dem QR-Code. Die Landingpage zeigt den Code an; der erste Code aus der App bestätigt die Einrichtung und meldet gleichzeitig an.

5. Schnittstellen

Methode	Pfad	Zweck
GET	`/`	Landingpage mit allen Ansichten
GET	`/welcome`	Geschützter Bereich (Weiterleitung ohne Session)
GET	`/api/health`	Status von Datenbank und SMTP-Konfiguration
GET	`/api/me`	Daten des angemeldeten Benutzers

Methode	Pfad	Zweck
POST	`/api/register`	Registrieren mit Modusauswahl
POST	`/api/login`	Login Schritt 1 (Passwort)
POST	`/api/verify-2fa`	Login Schritt 2 (E-Mail- oder TOTP-Code)
POST	`/api/forgot-password`	Reset-Code anfordern
POST	`/api/reset-password`	Neues Passwort mit Reset-Code setzen
POST	`/api/logout`	Abmelden und Session beenden

Fehlerfälle: 400 (Eingabe falsch), 401 (Zugangsdaten falsch), 403 (Login zeitlich gesperrt), 429 (zu viele Versuche, inklusive `Retry-After`), 500 (Serverfehler).

6. Passwort-Hashing

Das Passwort geht über HTTPS an den Server und wird dort mit **Argon2id** gehasht. Der Hash enthält das zufällige Salt und wird in der Datenbank gespeichert; das Klartext-Passwort wird nirgends abgelegt.

Eigenschaft	Wert
Verfahren	Argon2id (`$argon2id$v=19$…`)
Speicher	46 MiB (47104 KiB)
Durchläufe / Threads	1 / 1
Salt	zufällig pro Hash, im Hash-String enthalten
Dauer	ca. 50 ms pro Prüfung

Warum nicht im Browser hashen?

Ein Hash aus dem Browser ist selbst wieder ein vollwertiges Passwort: Wer ihn abfängt, kann sich damit direkt anmelden („Pass-the-Hash“). Ein einzelnes SHA-256 wäre außerdem viel zu schnell berechnet. Deshalb rechnet der Server – geschützt durch HTTPS – und verwendet mit Argon2id ein absichtlich langsames, speicherresistentes Verfahren. Das entspricht der OWASP-Empfehlung.

7. Sicherheit

Maßnahme	Umsetzung
Passwörter	Argon2id, zufälliges Salt, mindestens 8 Zeichen (serverseitig geprüft)
Session-Cookie	`HttpOnly`, `SameSite=Lax`, `Secure` (abschaltbar für lokale Tests), 7 Tage
Session-Token in der DB	nur als SHA-256-Hash – ein DB-Leak reicht nicht zur Übernahme
E-Mail-Codes	nur als HMAC-Signatur mit `APP_SECRET` gespeichert
TOTP-Secret	verschlüsselt (Fernet/AES-GCM), Schlüssel aus `APP_SECRET`
Kontoübernahme	Erneutes Registrieren ändert ein bestehendes Konto nur mit richtigem Passwort

Maßnahme	Umsetzung
Anti-Enumeration	gleiche Antwort bei unbekannter E-Mail; Dummy-Hash gleicht die Antwortzeit an
Header / CSP	`X-Frame-Options`, `nosniff`, `Referrer-Policy`, `script-src 'self'`
Container	Benutzer `app` (UID 10001), `cap_drop: ALL`, `no-new-privileges`

Brute-Force-Bremsen

Vorgang	Regel
E-Mail-Code / Reset-Code	nach 5 Fehlversuchen wird der Code ungültig (429)
TOTP-Code	nach 5 Fehlversuchen 15 Minuten gesperrt; jeder Zeitschritt nur einmal gültig
Login	20 Versuche / 15 min pro IP, 10 / 15 min pro E-Mail
Registrierung	10 Versuche / Stunde pro IP
Passwort-Reset	5 Anfragen / 15 min pro IP, 2 / Stunde pro E-Mail
Code-Prüfung	30 Versuche / 15 min pro IP

Der Limiter liegt im Speicher und gilt pro Prozess. Für mehrere Worker müsste der Zustand geteilt werden (zum Beispiel über Redis).

8. Tests und Werkzeuge

Die Testsuite läuft gegen eine temporäre SQLite-Datei und braucht weder MySQL noch einen Mailserver:

```
pip install -r requirements-dev.txt
pytest          # 30 Tests
ruff check .    # Stil- und Import-Prüfung
```

Abgedeckt sind Registrierung, Login, Sessions, E-Mail-2FA, TOTP inklusive Replay-Schutz, Passwort-Reset, die serverseitige Passwortregel, alle Rate-Limits, die Migration alter Datenbestände und die Auslieferung der Landingpage.

9. Inbetriebnahme

1. `cp .env.example .env` und bei Bedarf SMTP sowie `APP_SECRET` eintragen.
2. `docker compose up -d --build db web` startet Datenbank und Anwendung.
3. Die Anwendung ist unter `http://localhost:5000` erreichbar.
4. Für den öffentlichen Zugang das Cloudflare-Tunnel-Token eintragen und mit `--profile tunnel` starten.

Ohne SMTP-Konfiguration läuft die App im Dev-Modus: Der Code wird dann direkt auf der Seite angezeigt und zusätzlich ins Log geschrieben (`docker compose logs web`). Die ausführliche Anleitung steht in der `README.md`.

10. Ressourcenbedarf

Schätzwerte, nicht gemessen: insgesamt etwa 0,5–0,8 GB Arbeitsspeicher (MySQL ca. 400–600 MB, Web ca. 60–100 MB, cloudflared ca. 20–50 MB), praktisch keine CPU-Last im Leerlauf, ca. 1 GB für die Images. Empfohlen sind 1 vCPU und 2 GB RAM.

11. Bekannte Einschränkungen und nächste Schritte

- Der Rate-Limiter zählt pro Prozess; mehrere Worker bräuchten einen gemeinsamen Speicher (Redis).
- Schema-Änderungen passieren beim Start; für echte Migrationen wäre Alembic der nächste Schritt.
- Im Dev-Modus stehen Codes im Klartext im Log – praktisch für die Demo, nicht für den Produktivbetrieb.
- Die optionale Zeitsperre nutzt die Serverzeit des Containers (standardmäßig UTC).
- Es gibt noch keine Wiederherstellungscodes für verlorene Authenticator-Geräte.

12. Aufgabenteilung

Person	Vortragsteil
Person A	Idee, technischer Aufbau, die drei Login-Modi
Person B	Ablauf, Sicherheit, Live-Demo, Fazit